

Colas

Data Structures

Carlos Rojas-Sánchez

Universidad del Mar

Contenido

1. Representación de una cola

2. Operaciones con colas

3. Tipos de colas

Representación de una cola

¿Qué es una Cola (Queue)?

Una **cola** es una estructura de datos lineal que sigue el principio **FIFO** (*First In, First Out*: El primero en entrar es el primero en salir).

La Metáfora de la Fila de Personas

- Las personas se formen en una línea (por ejemplo, en el banco).
- El nuevo elemento se agrega siempre al **final** de la fila (*Enqueue*).
- La atención se realiza retirando siempre a la persona que está al **frente** (*Dequeue*).



Operaciones Principales de una Cola

A diferencia de las pilas (que solo operan en la cima), una cola opera en **ambos extremos**: el frente para salidas y el final para entradas.

Operaciones Esenciales

- **enqueue(elemento):** Inserta un elemento al final de la cola.
- **dequeue():** Extrae y elimina el elemento ubicado en el frente.
- **peek() / `front`():** Consulta el valor del frente sin eliminarlo.
- **`isEmpty`():** Verifica si la cola se encuentra vacía.

Formas de Representación en Memoria

Una cola puede implementarse computacionalmente de dos formas principales:

1. **Representación Estática (Arreglos Circulares)**
   - Utiliza un arreglo de tamaño fijo con dos índices (`frente` y `final`).
   - Se maneja de forma circular para aprovechar el espacio liberado.
   - **Riesgo:** Saturación o desbordamiento si se alcanza la capacidad máxima.

Formas de Representación en Memoria

2. **Representación Dinámica (Listas Enlazadas)**
   - Utiliza nodos en memoria dinámica (*Heap*).
   - Requiere dos punteros externos: uno a la `Cabeza/Frente` y otro a la `Cola/Final`.
   - Crece de manera flexible según la memoria disponible.

Definición en C: Nodo de la Cola

Definición estándar del nodo para una cola dinámica:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct NodoCola {
    int dato;
    struct NodoCola *siguiente;
} NodoCola;
```

Definición en C: Estructura de Control de la Cola

Estructura que mantiene los punteros al **frente** y al **final**:

```
typedef struct {
    NodoCola *frente; // Para extraer elementos (Dequeue)
    NodoCola *final;  // Para insertar elementos (Enqueue)
} ColaDinamica;

void inicializar_cola(ColaDinamica *c) {
    c->frente = NULL;
    c->final = NULL;
}
```

Ventaja clave: Mantener el puntero al `final` permite realizar inserciones en tiempo constante $\mathcal{O}(1)$ sin recorrer toda la lista.

Representación Gráfica: Cola Dinámica

Estructura visual de una cola enlazada con tres elementos:

$$\texttt{Cola.frente} \longrightarrow \texttt{[10 | *]} \longrightarrow \texttt{[20 | *]} \longrightarrow \texttt{[30 | NULL]} \longleftarrow \texttt{Cola.final}$$

- El nodo `10` es el próximo en salir (Frente).
- El nodo `30` fue el último en ingresar (Final).
- Las operaciones de inserción y extracción ocurren en extremos opuestos para garantizar el flujo FIFO.

Operaciones con colas

Operaciones Fundamentales

Las operaciones en una cola gestionan los elementos respetando estrictamente el principio FIFO, ocurriendo en extremos opuestos de la estructura.

Operaciones Esenciales

- **Enqueue (Encolar):** Inserta un elemento al final de la cola (`final`).
- **Dequeue (Desencolar):** Extrae y elimina el elemento ubicado en el frente (`frente`).
- **Peek / Front:** Consulta el valor del frente sin eliminarlo.
- **isEmpty:** Comprueba si la cola no contiene elementos.

Paso a Paso: Operación Enqueue (Encolar)

Agregar un elemento al final de la cola dinámica requiere:

Secuencia de Inserción

1. **Reservar Memoria:** Crear un nuevo nodo dinámicamente en el *Heap*.
2. **Asignar Valor:** Guardar el dato y establecer su puntero `siguiente = NULL`.
3. **Verificar Cola Vacía:** Si la cola está vacía (`frente == NULL`), tanto `frente` como `final` apuntarán al nuevo nodo.
4. **Enlazar y Actualizar:** Si no está vacía, conectar el nodo actual del `final` al nuevo nodo y actualizar el puntero `final`.

Código: Operación Enqueue

Implementación de la inserción al final usando una cola dinámica:

```
void enqueue(ColaDinamica *cola, int valor) {
    NodoCola *nuevo = (NodoCola*)malloc(sizeof(NodoCola));
    if (nuevo == NULL) {
        printf("Error: Memoria llena (Overflow)\n");
        return;
    }
    nuevo->dato = valor;
    nuevo->siguiente = NULL;

    if (cola->frente == NULL) { // Si la cola está vacía
        cola->frente = nuevo;
        cola->final = nuevo;
    } else {
        cola->final->siguiente = nuevo; // Enlazar al final actual
        cola->final = nuevo;            // Actualizar el puntero final
    }
    printf("Enqueue: %d\n", valor);
}
```

Paso a Paso: Operación Dequeue (Desencolar)

Extraer el elemento del frente de la cola de manera segura:

Secuencia de Extracción

1. **Validación:** Comprobar que la cola no esté vacía (`frente != NULL`) para evitar errores de tipo *Underflow*.
2. **Guardar Referencia:** Asignar un puntero temporal al nodo del frente (`temp = cola->frente`).
3. **Mover Frente:** Actualizar `cola->frente = cola->frente->siguiente`.
4. **Caso Límite:** Si la cola queda vacía tras la extracción, hacer `cola->final = NULL`.
5. **Liberar y Retornar:** Guardar el valor, liberar la memoria con `free(temp)` y retornar el dato.

Código: Operación Dequeue

Implementación de la extracción en el frente con manejo de cola vacía:

```
int dequeue(ColaDinamica *cola) {
    if (cola->frente == NULL) {
        printf("Error: Cola vacia (Underflow)\n");
        return -1;
    }
    NodoCola *temp = cola->frente;
    int valor_extraido = temp->dato;

    cola->frente = temp->siguiente; // Avanzar el frente

    if (cola->frente == NULL) {     // Si quedó vacía, actualizar final
        cola->final = NULL;
    }

    free(temp);                     // Liberar memoria del nodo extraído
    return valor_extraido;
}
```

Operaciones Auxiliares: Peek e isEmpty

Funciones complementarias para consultar el estado de la cola:

1. Verificar si está vacía (`isEmpty`):

```
int isEmpty(ColaDinamica *cola) {
    return (cola->frente == NULL); // Retorna 1 si está vacía, 0 si tiene elementos
}
```

2. Consultar el Frente sin extraer (`peek`):

```
int peek(ColaDinamica *cola) {
    if (isEmpty(cola)) {
        printf("La cola esta vacia.\n");
        return -1;
    }
    return cola->frente->dato; // Retorna el valor del primer elemento
}
```

Tipos de colas

Clasificación de las Colas

Dependiendo de las restricciones de inserción y eliminación, existen diversas variantes especializadas de colas.

Variantes Principales

- **Colas Simples (Lineales):** Inserciones al final y extracciones al frente en un solo sentido.
- **Colas Circulares:** Conectan el último elemento con el primero para optimizar el almacenamiento en arreglos estáticos.
- **Colas Dobles (Deque - Double Ended Queue):** Permiten inserciones y eliminaciones tanto en el frente como en el final.

1. Colas Simples

Es la estructura tradicional y básica bajo el modelo estricto FIFO.

Características

- Los elementos entran por el `final` y salen por el `frente`.
- **Desventaja en arreglos estáticos:** Una vez que el índice `final` alcanza el tamaño máximo del arreglo, no se pueden reusar las posiciones vacías dejadas al frente por extracciones previas, a menos que se reacomoden los elementos.

`Frente` $\longrightarrow$ `[ 10 ]` $\longrightarrow$ `[ 20 ]` $\longrightarrow$ `[ 30 ]` $\longleftarrow$ `Final`

2. Colas Circulares

Resuelven el problema de desperdicio de espacio en arreglos estáticos conectando lógicamente el último elemento con el primero en forma de anillo.

Características

- El índice `final` puede regresar a la posición 0 del arreglo si hay espacio disponible al inicio.
- Utiliza operaciones aritméticas modulares para el avance de índices: `indice = (indice + 1) % MAX`.
- Evita la necesidad de desplazar elementos en memoria.

Estructura en C: Cola Circular Estática

Implementación típica utilizando un arreglo de tamaño fijo con control circular:

```
#define MAX 5

typedef struct {
    int elementos[MAX];
    int frente;
    int final;
    int tamano;
} ColaCircular;

void inicializar_circular(ColaCircular *c) {
    c->frente = 0;
    c->final = -1;
    c->tamano = 0;
}
```

3. Colas Dobles (Deque - Double Ended Queue)

Estructura flexible donde las inserciones y eliminaciones se pueden realizar tanto en el **frente** como en el **final**.

Variantes de Deque

- **Deque con entrada restringida:** Se inserta solo por un extremo, pero se elimina por ambos.
- **Deque con salida restringida:** Se inserta por ambos extremos, pero se elimina por uno solo.

$$\texttt{Frente (Entrada/Salida)} \rightleftarrows \texttt{[10]} \rightleftarrows \texttt{[20]} \rightleftarrows \texttt{Final (Entrada/Salida)}$$

Cuadro Comparativo de Tipos de Colas

Tipo de Cola	Puntos de Operación	Ventaja Principal
Simple	Entrada al final, salida al frente	Implementación directa y sencilla.
Circular	Arreglo en anillo (Modular)	Reutiliza espacio vacío en memoria estática.
Doble (Deque)	Ambos extremos (Frente y Final)	Máxima flexibilidad operativa (FIFO y LIFO).

Aplicaciones de las colas

Aplicaciones Prácticas de las Colas

Gracias al principio FIFO (*First In, First Out*), las colas son fundamentales en sistemas donde los recursos deben gestionarse en orden estrictamente cronológico o de llegada.

Áreas de Aplicación Principal

- **Sistemas de Impresión:** Gestión de documentos en cola de impresión (*Print Queue*).
- **Planificación de Procesos (CPU):** Algoritmos de despacho de tareas en sistemas operativos.
- **Redes y Comunicaciones:** Búferes de paquetes en routers y switches.
- **Algoritmos de Grafos:** Recorridos en anchura (BFS - *Breadth-First Search*).

1. Gestión de Impresión (Print Queues)

Cuando múltiples usuarios o aplicaciones envían documentos a una impresora en red, estos no pueden imprimirse al mismo tiempo.

Funcionamiento

- El sistema operativo intercepta los trabajos de impresión y los almacena en una cola en orden de llegada.
- El primer documento en enviarse es el primero en ser impreso (**FIFO**).
- Garantiza que no haya solapamiento de datos y que cada usuario reciba sus páginas en orden.

2. Planificación de Procesos en Sistemas Operativos

Los sistemas operativos multitarea gestionan el procesador y los dispositivos de E/S mediante colas de espera.

Tipos de Planificación

- **Cola de Listos (Ready Queue):** Procesos cargados en memoria principal esperando un turno de CPU.
- **Colas de Dispositivos (Device Queues):** Procesos esperando la disponibilidad de recursos como discos duros, tarjetas de red o puertos USB.
- **FIFO / Round-Robin:** Asignación equitativa de turnos de procesamiento en forma de cola circular.

3. Redes y Manejo de Tráfico (Buffering)

En telecomunicaciones y transmisión de datos, las colas evitan la pérdida de información cuando hay congestión.

Búferes de Paquetes

- Los routers y conmutadores almacenan temporalmente los paquetes de red en colas de tipo FIFO antes de retransmitirlos.
- Si la tasa de llegada supera la capacidad de salida, la cola regula el flujo (control de congestión).
- Utilizado en streaming de audio/video para evitar interrupciones por variaciones en la red (*jitter*).

4. Algoritmos de Grafos: Recorrido en Anchura (BFS)

En ciencias de la computación, las colas permiten explorar estructuras de grafos y árboles nivel por nivel.

Algoritmo BFS (Breadth-First Search)

- Se visita un nodo inicial y se registran todos sus vecinos adyacentes pendientes.
- Se utiliza una **Cola** para almacenar los nodos descubiertos en orden de proximidad.
- Garantiza encontrar el camino más corto en grafos no ponderados.

Resumen de Aplicaciones y Ventajas de las Colas

Aplicación	Dominio	Propósito Principal
Cola de Impresión	Sistemas Operativos	Procesar documentos en orden exacto de llegada.
Planificación CPU	Arquitectura de Computadores	Distribuir turnos de ejecución (FIFO/RR).
Búferes de Red	Telecomunicaciones	Regular flujos de datos y prevenir pérdida de paquetes.
Recorrido BFS	Algoritmia y Grafos	Explorar nodos de un grafo por niveles de distancia.